

Web Sites Monitoring System (WSMS)

COMPLETE SOURCE CODE IMPLEMENTATION BOOK

A comprehensive, compiled reference of all core backend Java services, DTOs, configurations, rate limiting filters, and frontend views that implement the WSMS platform.

Author: Manikanta & Antigravity AI Co-Pilot
Platform Version: 1.0.0 | Release Date: June 2026

Table of Contents

File: backend/pom.xml Page 3

Maven Dependencies Configuration
Description: Maven Dependencies Configuration

backend/src/main/resources/application.yml Page 5

```
1: <?xml version="1.0" encoding="UTF-8"?>
2: <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema ...
3:     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/ma ...
4:     <modelVersion>4.0.0</modelVersion>
```

backend/src/main/java/com/wsms/WsmsApplication.java Page 7

```
Application Main Entry Point
6:     <groupId>org.springframework.boot</groupId>
7:     <artifactId>spring-boot-starter-parent</artifactId>
8:     <version>3.2.5</version>
```

backend/src/main/java/com/wsms/config/SecurityConfig.java Page 8

```
Spring Security Configuration
9:     <relativePath/> <!-- lookup parent from repository -->
10:    </parent>
```

backend/src/main/java/com/wsms/config/RateLimitingFilter.java Page 10

```
12:    <artifactId>wsms</artifactId>
13:    <version>1.0.0</version>
14:    <name>Web Sites Monitoring System</name>
```

backend/src/main/java/com/wsms/config/JwtAuthenticationFilter.java Page 12

```
15: <!-- Lightweight uptime and perimeter monitoring platform --> </description>
16: <!-- Bearer Token Filter
17:    <properties>
18:    <java.version>17</java.version>
19:    </properties>
```

backend/src/main/java/com/wsms/config/AiConfig.java Page 14

```
Spring AI & VectorStore Bean Config
20:    <dependencyManagement>
```

backend/src/main/java/com/wsms/config/DatabaseMigrationRunner.java Page 15

```
23:    <dependency>
24:    <groupId>org.springframework.ai</groupId>
25:    <artifactId>spring-ai-bom</artifactId>
26:    <version>0.8.1</version>
```

backend/src/main/java/com/wsms/controller/AuthenticationController.java ... Page 18

```
27:    <type>pom</type>
28:    <scope>import</scope>
29:    </dependency>
30:    </dependencies>
```

backend/src/main/java/com/wsms/dto/LoginRequestDto.java Page 20

```
31:    <dependencyManagement>
32:    <dependency>
```

backend/src/main/java/com/wsms/dto/RegisterRequestDto.java Page 21

```
34:    <!-- Spring Boot Web Starter for REST APIs -->
35:    <dependency>
```

backend/src/main/java/com/wsms/service/AuthenticationService.java Page 22

```
38:    </dependency>
39:    </dependencies>
```

backend/src/main/java/com/wsms/service/EncryptionService.java Page 25

```
40:    <!-- Spring Boot JPA Starter for Data Access -->
41:    <dependency>
42:    <groupId>org.springframework.boot</groupId>
43:    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

backend/src/main/java/com/wsms/service/RecaptchaService.java Page 26

```
44:    </dependency>
45:    </dependencies>
46:    <!-- Spring Boot Validation Starter -->
```

backend/src/main/java/com/wsms/service/RagService.java Page 28

```
49:    <artifactId>spring-boot-starter-validation</artifactId>
50:    </dependency>
```

backend/src/main/java/com/wsms/service/AIService.java Page 31

```
51:    <!-- PostgreSQL Database Driver -->
52:    <dependency>
53:    <groupId>org.postgresql</groupId>
```

frontend/src/views/Login.vue Page 34

```
55:    <artifactId>postgresql</artifactId>
56:    <scope>runtime</scope>
57:    </dependency>
```

```
58:    </dependencies>
```

```
59:         <!-- Spring Boot Security Starter -->
60:         <dependency>
61:             <groupId>org.springframework.boot</groupId>
62:             <artifactId>spring-boot-starter-security</artifactId>
63:         </dependency>
64:
65:         <!-- Auth0 JWT for token signing and verification -->
66:         <dependency>
67:             <groupId>com.auth0</groupId>
68:             <artifactId>java-jwt</artifactId>
69:             <version>4.4.0</version>
70:         </dependency>
71:
72:         <!-- Spring Boot Test Starter -->
73:         <dependency>
74:             <groupId>org.springframework.boot</groupId>
75:             <artifactId>spring-boot-starter-test</artifactId>
76:             <scope>test</scope>
77:         </dependency>
78:
79:         <!-- Spring AI Ollama Starter -->
80:         <dependency>
81:             <groupId>org.springframework.ai</groupId>
82:             <artifactId>spring-ai-ollama-spring-boot-starter</artifactId>
83:         </dependency>
84:     </dependencies>
85:
86:     <build>
87:         <plugins>
88:             <plugin>
89:                 <groupId>org.springframework.boot</groupId>
90:                 <artifactId>spring-boot-maven-plugin</artifactId>
91:             </plugin>
92:         </plugins>
93:     </build>
94:
95:     <repositories>
96:         <repository>
97:             <id>spring-milestones</id>
98:             <name>Spring Milestones</name>
99:             <url>https://repo.spring.io/milestone</url>
100:             <snapshots>
101:                 <enabled>false</enabled>
102:             </snapshots>
103:         </repository>
104:     </repositories>
105: </project>
```

File: backend/src/main/resources/application.yml

Description: Spring Boot Properties Config

```
1: server:
2:   port: 8081
3:
4: spring:
5:   application:
6:     name: wsms
7:
8:   # PostgreSQL Connection Settings
9:   datasource:
10:    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/wsms}
11:    username: ${SPRING_DATASOURCE_USERNAME:postgres}
12:    password: ${SPRING_DATASOURCE_PASSWORD:mani2834}
13:    driver-class-name: org.postgresql.Driver
14:
15:   # Hibernate JPA configurations
16:   jpa:
17:     database-platform: org.hibernate.dialect.PostgreSQLDialect
18:     hibernate:
19:       ddl-auto: update
20:       show-sql: false
21:       properties:
22:         hibernate:
23:           format_sql: true
24:           jdbc:
25:             time_zone: UTC
26:
27:   # AI Monitoring Assistant Ollama settings
28:   ai:
29:     ollama:
30:       base-url: ${OLLAMA_BASE_URL:http://localhost:11434}
31:       chat:
32:         model: qwen2.5:1.5b
33:         options:
34:           model: qwen2.5:1.5b
35:           temperature: 0.2
36:           num-predict: 1024
37:       embedding:
38:         model: all-minilm
39:
40:
41:   # System Probing Configuration
42:   wsms:
43:     probing:
44:       pool-size: 10
45:       timeout-ms: 5000
46:       retry-count: 3
47:       retry-delay-ms: 10000 # Delay between rapid retries in milliseconds
48:
49:   # Logging Configuration
50:   logging:
51:     level:
52:       root: INFO
53:       com.wsms: DEBUG
54:       org.hibernate.SQL: ERROR
55:     pattern:
56:       console: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
57:
58:   # Bot Protection Settings
```

```
59: google:
60:   recaptcha:
61:     secret: ${RECAPTCHA_SECRET:6LeIxAcTAAAAAGG-vFI1TnX0AGzIbKgEZXh1QI3r}
62:     site-key: ${RECAPTCHA_SITE_KEY:6LeIxAcTAAAAAJcZVRqyHh71UMTErE_Yth5aqaf1}
63:     threshold: 0.5
64:
65: cloudflare:
66:   turnstile:
67:     secret: ${TURNSTILE_SECRET:1x000000000000000000000000000000AA}
68:     site-key: ${TURNSTILE_SITE_KEY:1x000000000000000000000000AA}
69:
70:
```

File: backend/src/main/java/com/wsms/WsmsApplication.java

Description: Application Main Entrypoint

```
1: package com.wsms;
2:
3: import org.springframework.boot.SpringApplication;
4: import org.springframework.boot.autoconfigure.SpringBootApplication;
5: import org.springframework.scheduling.annotation.EnableScheduling;
6:
7: @SpringBootApplication
8: @EnableScheduling
9: public class WsmsApplication {
10:
11:     public static void main(String[] args) {
12:         SpringApplication.run(WsmsApplication.class, args);
13:         System.out.println("=====");
14:         System.out.println("  Web Sites Monitoring System (WSMS) is online!  ");
15:         System.out.println("  Access Dashboard: http://localhost:8081/      ");
16:         System.out.println("=====");
17:     }
18: }
```

File: backend/src/main/java/com/wsms/config/SecurityConfig.java

Description: Spring Security Configuration

```
1: package com.wsms.config;
2:
3: import com.wsms.service.CustomUserDetailsService;
4: import org.springframework.context.annotation.Bean;
5: import org.springframework.context.annotation.Configuration;
6: import org.springframework.security.authentication.AuthenticationManager;
7: import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
8: import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfigurat ...
9: import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurit ...
10: import org.springframework.security.config.annotation.web.builders.HttpSecurity;
11: import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
12: import org.springframework.security.config.http.SessionCreationPolicy;
13: import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
14: import org.springframework.security.crypto.password.PasswordEncoder;
15: import org.springframework.security.web.SecurityFilterChain;
16: import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
17: import org.springframework.web.cors.CorsConfiguration;
18: import org.springframework.web.cors.CorsConfigurationSource;
19: import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
20:
21: import java.util.Arrays;
22: import java.util.List;
23:
24: @Configuration
25: @EnableWebSecurity
26: @EnableMethodSecurity
27: public class SecurityConfig {
28:
29:     private final JwtAuthenticationFilter jwtAuthFilter;
30:     private final CustomUserDetailsService userDetailsService;
31:     private final RateLimitingFilter rateLimitingFilter;
32:
33:     public SecurityConfig(
34:         JwtAuthenticationFilter jwtAuthFilter,
35:         CustomUserDetailsService userDetailsService,
36:         RateLimitingFilter rateLimitingFilter
37:     ) {
38:         this.jwtAuthFilter = jwtAuthFilter;
39:         this.userDetailsService = userDetailsService;
40:         this.rateLimitingFilter = rateLimitingFilter;
41:     }
42:
43:     @Bean
44:     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
45:         http
46:             .cors(cors -> cors.configurationSource(corsConfigurationSource()))
47:             .csrf(csrf -> csrf.disable())
48:             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy. ...
49:             .authorizeHttpRequests(auth -> auth
50:                 .requestMatchers("/api/auth/**").permitAll()
51:                 .requestMatchers("/api/ai/**").permitAll()
52:                 .requestMatchers("/api/**").authenticated()
53:                 .anyRequest().permitAll()
54:             )
55:             .headers(headers -> headers
56:                 .xssProtection(xss -> xss.disable())
57:                 .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self' *; conn ...
58:                 .httpStrictTransportSecurity(hsts -> hsts
```

```
59:         .includeSubDomains(true)
60:         .maxAgeInSeconds(31536000)
61:     )
62: )
63: .authenticationProvider(authenticationProvider())
64: .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
65: .addFilterBefore(rateLimitingFilter, JwtAuthenticationFilter.class);
66:
67:     return http.build();
68: }
69:
70: @Bean
71: public PasswordEncoder passwordEncoder() {
72:     return new BCryptPasswordEncoder();
73: }
74:
75: @Bean
76: public DaoAuthenticationProvider authenticationProvider() {
77:     DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
78:     authProvider.setUserDetailsService(userDetailsService);
79:     authProvider.setPasswordEncoder(passwordEncoder());
80:     return authProvider;
81: }
82:
83: @Bean
84: public AuthenticationManager authenticationManager(AuthenticationConfiguration config) thr ...
85:     return config.getAuthenticationManager();
86: }
87:
88: @Bean
89: public CorsConfigurationSource corsConfigurationSource() {
90:     CorsConfiguration configuration = new CorsConfiguration();
91:     configuration.setAllowedOriginPatterns(List.of("*"));
92:     configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS ...
93:     configuration.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type", "Cache- ...
94:     configuration.setAllowCredentials(true);
95:     UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
96:     source.registerCorsConfiguration("/*", configuration);
97:     return source;
98: }
99: }
```

File: backend/src/main/java/com/wsms/config/RateLimitingFilter.java

Description: IP-Based Token Bucket Limiter

```
1: package com.wsms.config;
2:
3: import jakarta.servlet.*;
4: import jakarta.servlet.http.HttpServletRequest;
5: import jakarta.servlet.http.HttpServletResponse;
6: import org.slf4j.Logger;
7: import org.slf4j.LoggerFactory;
8: import org.springframework.http.HttpStatus;
9: import org.springframework.stereotype.Component;
10:
11: import java.io.IOException;
12: import java.util.concurrent.ConcurrentHashMap;
13: import java.util.concurrent.TimeUnit;
14:
15: @Component
16: public class RateLimitingFilter implements Filter {
17:
18:     private static final Logger log = LoggerFactory.getLogger(RateLimitingFilter.class);
19:
20:     private static final int MAX_REQUESTS = 5;
21:     private static final long TIME_WINDOW_MS = TimeUnit.MINUTES.toMillis(1); // 1 minute windo ...
22:
23:     // In-memory cache tracking client IP requests
24:     private final ConcurrentHashMap<String, TokenBucket> ipCache = new ConcurrentHashMap<>();
25:
26:     @Override
27:     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
28:         throws IOException, ServletException {
29:
30:         if (request instanceof HttpServletRequest httpServletRequest && response instanceof Ht ...
31:             String clientId = getClientIp(httpServletRequest);
32:             String path = httpServletRequest.getRequestURI();
33:
34:             // Only enforce rate limits on login/auth requests to prevent DDoS/abuse of auth e ...
35:             if (path.startsWith("/api/auth/")) {
36:                 TokenBucket bucket = ipCache.computeIfAbsent(clientId, k -> new TokenBucket(MA ...
37:
38:                 if (!bucket.tryConsume()) {
39:                     log.warn("Rate limit breached for IP: {} on URI: {}", clientId, path);
40:                     httpServletResponse.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
41:                     httpServletResponse.setContentType("application/json");
42:                     httpServletResponse.getWriter().write("{\"message\": \"Too many requests. ...
43:                     return;
44:                 }
45:             }
46:         }
47:
48:         chain.doFilter(request, response);
49:     }
50:
51:     private String getClientIp(HttpServletRequest request) {
52:         String xfHeader = request.getHeader("X-Forwarded-For");
53:         if (xfHeader == null || xfHeader.isBlank()) {
54:             return request.getRemoteAddr();
55:         }
56:         return xfHeader.split(",")[0].trim();
57:     }
58: }
```

```
59:     private static class TokenBucket {
60:         private final int maxTokens;
61:         private final long refillIntervalMs;
62:         private double tokens;
63:         private long lastRefillTime;
64:
65:         public TokenBucket(int maxTokens, long refillIntervalMs) {
66:             this.maxTokens = maxTokens;
67:             this.refillIntervalMs = refillIntervalMs;
68:             this.tokens = maxTokens;
69:             this.lastRefillTime = System.currentTimeMillis();
70:         }
71:
72:         public synchronized boolean tryConsume() {
73:             refill();
74:             if (tokens >= 1.0) {
75:                 tokens -= 1.0;
76:                 return true;
77:             }
78:             return false;
79:         }
80:
81:         private void refill() {
82:             long now = System.currentTimeMillis();
83:             long elapsedTime = now - lastRefillTime;
84:
85:             // Add tokens proportionally to elapsed time
86:             double tokensToAdd = ((double) elapsedTime / refillIntervalMs) * maxTokens;
87:             if (tokensToAdd > 0) {
88:                 tokens = Math.min(maxTokens, tokens + tokensToAdd);
89:                 lastRefillTime = now;
90:             }
91:         }
92:     }
93: }
```

File: backend/src/main/java/com/wsms/config/JwtAuthenticationFilter.java

Description: JWT Bearer Token Filter

```
1: package com.wsms.config;
2:
3: import com.wsms.service.CustomUserDetailsService;
4: import com.wsms.service.JwtService;
5: import jakarta.servlet.FilterChain;
6: import jakarta.servlet.ServletException;
7: import jakarta.servlet.http.HttpServletRequest;
8: import jakarta.servlet.http.HttpServletResponse;
9: import org.springframework.lang.NonNull;
10: import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
11: import org.springframework.security.core.context.SecurityContextHolder;
12: import org.springframework.security.core.userdetails.UserDetails;
13: import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
14: import org.springframework.stereotype.Component;
15: import org.springframework.web.filter.OncePerRequestFilter;
16:
17: import java.io.IOException;
18:
19: @Component
20: public class JwtAuthenticationFilter extends OncePerRequestFilter {
21:
22:     private final JwtService jwtService;
23:     private final CustomUserDetailsService userDetailsService;
24:
25:     public JwtAuthenticationFilter(JwtService jwtService, CustomUserDetailsService userDetailsService ...
26:         this.jwtService = jwtService;
27:         this.userDetailsService = userDetailsService;
28:     }
29:
30:     @Override
31:     protected void doFilterInternal(
32:         @NonNull HttpServletRequest request,
33:         @NonNull HttpServletResponse response,
34:         @NonNull FilterChain filterChain
35:     ) throws ServletException, IOException {
36:         final String authHeader = request.getHeader("Authorization");
37:         final String jwt;
38:         final String username;
39:
40:         if (authHeader == null || !authHeader.startsWith("Bearer ")) {
41:             filterChain.doFilter(request, response);
42:             return;
43:         }
44:
45:         jwt = authHeader.substring(7);
46:         try {
47:             username = jwtService.extractUsername(jwt);
48:
49:             if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
50:                 UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);
51:
52:                 if (jwtService.isTokenValid(jwt, userDetails)) {
53:                     UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthen ...
54:                         userDetails,
55:                         null,
56:                         userDetails.getAuthorities()
57:                     );
58:                     authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(req ...
```

```
59:             SecurityContextHolder.getContext().setAuthentication(authToken);
60:         }
61:     }
62: } catch (Exception e) {
63:     // Ignore verification failure, just don't set authentication context
64: }
65:
66: filterChain.doFilter(request, response);
67: }
68: }
```

File: backend/src/main/java/com/wsms/config/AiConfig.java

Description: Spring AI & VectorStore Bean Config

```
1: package com.wsms.config;
2:
3: import org.springframework.ai.embedding.EmbeddingClient;
4: import org.springframework.ai.ollama.OllamaChatClient;
5: import org.springframework.ai.ollama.OllamaEmbeddingClient;
6: import org.springframework.ai.ollama.api.OllamaApi;
7: import org.springframework.ai.ollama.api.OllamaOptions;
8: import org.springframework.ai.vectorstore.SimpleVectorStore;
9: import org.springframework.beans.factory.annotation.Value;
10: import org.springframework.context.annotation.Bean;
11: import org.springframework.context.annotation.Configuration;
12: import org.springframework.http.client.SimpleClientHttpRequestFactory;
13: import org.springframework.web.client.RestClient;
14:
15: @Configuration
16: public class AiConfig {
17:
18:     @Value("${spring.ai.ollama.base-url:http://localhost:11434}")
19:     private String baseUrl;
20:
21:     @Value("${spring.ai.ollama.chat.model:qwen2.5:1.5b}")
22:     private String modelName;
23:
24:     @Value("${spring.ai.ollama.embedding.model:all-minilm}")
25:     private String embeddingModelName;
26:
27:     @Bean
28:     public OllamaApi ollamaApi() {
29:         SimpleClientHttpRequestFactory requestFactory = new SimpleClientHttpRequestFactory();
30:         requestFactory.setConnectTimeout(60000); // 60 seconds connection timeout
31:         requestFactory.setReadTimeout(120000); // 120 seconds read timeout
32:
33:         RestClient.Builder restClientBuilder = RestClient.builder().requestFactory(requestFact ...
34:         return new OllamaApi(baseUrl, restClientBuilder);
35:     }
36:
37:     @Bean
38:     public OllamaChatClient chatClient(OllamaApi ollamaApi) {
39:         OllamaOptions options = OllamaOptions.create().withModel(modelName);
40:         return new OllamaChatClient(ollamaApi).withDefaultOptions(options);
41:     }
42:
43:     @Bean
44:     public OllamaEmbeddingClient embeddingClient(OllamaApi ollamaApi) {
45:         OllamaOptions options = OllamaOptions.create().withModel(embeddingModelName);
46:         return new OllamaEmbeddingClient(ollamaApi).withDefaultOptions(options);
47:     }
48:
49:     @Bean
50:     public SimpleVectorStore vectorStore(EmbeddingClient embeddingClient) {
51:         return new SimpleVectorStore(embeddingClient);
52:     }
53: }
54:
```

File: backend/src/main/java/com/wsms/config/DatabaseMigrationRunner.java

Description: Database Seeding & Schema Migrations

```
1: package com.wsms.config;
2:
3: import org.slf4j.Logger;
4: import org.slf4j.LoggerFactory;
5: import org.springframework.beans.factory.annotation.Autowired;
6: import org.springframework.boot.CommandLineRunner;
7: import org.springframework.jdbc.core.JdbcTemplate;
8: import org.springframework.stereotype.Component;
9: import org.springframework.ai.vectorstore.SimpleVectorStore;
10:
11: @Component
12: public class DatabaseMigrationRunner implements CommandLineRunner {
13:
14:     private static final Logger log = LoggerFactory.getLogger(DatabaseMigrationRunner.class);
15:
16:     private final JdbcTemplate jdbcTemplate;
17:     private final SimpleVectorStore vectorStore;
18:
19:     @Autowired
20:     public DatabaseMigrationRunner(JdbcTemplate jdbcTemplate, SimpleVectorStore vectorStore) {
21:         this.jdbcTemplate = jdbcTemplate;
22:         this.vectorStore = vectorStore;
23:     }
24:
25:
26:     @Override
27:     public void run(String... args) throws Exception {
28:         log.info("Running database migration check to verify websites table columns...");
29:
30:         String[] alterStatements = {
31:             "ALTER TABLE websites ADD COLUMN IF NOT EXISTS dns_lookup_time DOUBLE PRECISION DE ...",
32:             "ALTER TABLE websites ADD COLUMN IF NOT EXISTS ssl_issuer VARCHAR(255)",
33:             "ALTER TABLE websites ADD COLUMN IF NOT EXISTS protocol VARCHAR(10)",
34:             "ALTER TABLE websites ADD COLUMN IF NOT EXISTS ssl_expiry_threshold INT DEFAULT 30 ...",
35:             "ALTER TABLE websites ADD COLUMN IF NOT EXISTS dns_lookup_threshold DOUBLE PRECISI ...",
36:             "ALTER TABLE websites ADD COLUMN IF NOT EXISTS ewma_threshold DOUBLE PRECISION DEF ...",
37:             "ALTER TABLE users ADD COLUMN IF NOT EXISTS failed_attempts INT DEFAULT 0 NOT NULL ...",
38:             "ALTER TABLE users ADD COLUMN IF NOT EXISTS lockout_time TIMESTAMP WITH TIME ZONE"
39:         };
40:
41:         for (String sql : alterStatements) {
42:             try {
43:                 jdbcTemplate.execute(sql);
44:                 log.debug("Executed query successfully: {}", sql);
45:             } catch (Exception e) {
46:                 log.warn("Query execution failed or skipped (may already exist): {}. Error: {} ...", sql, e.getMessage());
47:             }
48:         }
49:         log.info("Database migration check completed.");
50:
51:         log.info("Verifying security and user database schemas...");
52:         try {
53:             // 1. Create Roles table
54:             jdbcTemplate.execute("CREATE TABLE IF NOT EXISTS roles (" +
55:                 "id SERIAL PRIMARY KEY, " +
56:                 "name VARCHAR(20) NOT NULL UNIQUE)");
57:
58:             // 2. Create Users table
```

```
59:         jdbcTemplate.execute("CREATE TABLE IF NOT EXISTS users (" +
60:             "id SERIAL PRIMARY KEY, " +
61:             "username VARCHAR(50) NOT NULL UNIQUE, " +
62:             "email VARCHAR(100) NOT NULL UNIQUE, " +
63:             "password VARCHAR(100) NOT NULL, " +
64:             "enabled BOOLEAN DEFAULT TRUE NOT NULL, " +
65:             "failed_attempts INT DEFAULT 0 NOT NULL, " +
66:             "lockout_time TIMESTAMP WITH TIME ZONE, " +
67:             "created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL)") ...
68:
69: // 3. Create User-Roles join table
70: jdbcTemplate.execute("CREATE TABLE IF NOT EXISTS user_roles (" +
71:     "user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE, " +
72:     "role_id INT NOT NULL REFERENCES roles(id) ON DELETE CASCADE, " +
73:     "PRIMARY KEY (user_id, role_id))");
74:
75: // 4. Create Refresh Tokens table
76: jdbcTemplate.execute("CREATE TABLE IF NOT EXISTS refresh_tokens (" +
77:     "id SERIAL PRIMARY KEY, " +
78:     "token VARCHAR(255) NOT NULL UNIQUE, " +
79:     "user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE, " +
80:     "expiry_date TIMESTAMP WITH TIME ZONE NOT NULL)");
81:
82: // 5. Create Security Runbooks table for RAG
83: jdbcTemplate.execute("CREATE TABLE IF NOT EXISTS security_runbooks (" +
84:     "id SERIAL PRIMARY KEY, " +
85:     "title VARCHAR(150) NOT NULL UNIQUE, " +
86:     "category VARCHAR(50) NOT NULL, " +
87:     "content TEXT NOT NULL, " +
88:     "created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL)") ...
89:
90: log.info("Security database schemas verified successfully.");
91:
92: // 6. Seed default roles
93: jdbcTemplate.execute("INSERT INTO roles (name) VALUES ('ROLE_ADMIN') ON CONFLICT ( ...
94: jdbcTemplate.execute("INSERT INTO roles (name) VALUES ('ROLE_OPERATOR') ON CONFLIC ...
95: jdbcTemplate.execute("INSERT INTO roles (name) VALUES ('ROLE_VIEWER') ON CONFLICT ...
96:
97: // 7. Seed default RAG runbooks
98: jdbcTemplate.update("INSERT INTO security_runbooks (title, category, content) VALU ...
99:     "('SSL Expiry Playbook', 'SSL', 'If an SSL certificate is close to expirat ...
100:     "ON CONFLICT (title) DO NOTHING");
101:
102: jdbcTemplate.update("INSERT INTO security_runbooks (title, category, content) VALU ...
103:     "('Outage Recovery', 'Outage', 'For HTTP 500/502/503 errors, check the tar ...
104:     "ON CONFLICT (title) DO NOTHING");
105:
106: jdbcTemplate.update("INSERT INTO security_runbooks (title, category, content) VALU ...
107:     "('Response Time Spikes', 'Latency', 'When EWMA response time exceeds the ...
108:     "ON CONFLICT (title) DO NOTHING");
109:
110: // 6. Seed default administrator user
111: String hashedPassword = new org.springframework.security.crypto.bcrypt.BCryptPassw ...
112: Integer count = jdbcTemplate.queryForObject("SELECT COUNT(*) FROM users WHERE user ...
113: if (count == null || count == 0) {
114:     jdbcTemplate.update("INSERT INTO users (username, email, password, enabled, fa ...
115: } else {
116:     // Force update the password in case it was altered or old, and reset failed a ...
117:     jdbcTemplate.update("UPDATE users SET password = ?, enabled = true, failed_att ...
118: }
119:
120: Integer userId = jdbcTemplate.queryForObject("SELECT id FROM users WHERE username ...
121: Integer roleId = jdbcTemplate.queryForObject("SELECT id FROM roles WHERE name = 'R ...
```

```
122:
123:         if (userId != null && roleId != null) {
124:             jdbcTemplate.update("INSERT INTO user_roles (user_id, role_id) VALUES (?, ?) 0 ...
125:         }
126:         log.info("Default administrator account 'admin' created or updated successfully wi ...
127:
128:         // Print existing users for verification
129:         jdbcTemplate.query("SELECT username, email FROM users", (rs, rowNum) -> {
130:             log.info("User in Database: {} (email: {})", rs.getString("username"), rs.getS ...
131:             return null;
132:         });
133:
134:         log.info("Initializing SimpleVectorStore with security runbooks...");
135:         java.util.List<org.springframework.ai.document.Document> documents = jdbcTemplate. ...
136:             "SELECT title, category, content FROM security_runbooks",
137:             (rs, rowNum) -> {
138:                 java.util.Map<String, Object> metadata = new java.util.HashMap<>();
139:                 metadata.put("title", rs.getString("title"));
140:                 metadata.put("category", rs.getString("category"));
141:                 return new org.springframework.ai.document.Document(rs.getString("content" ...
142:             }
143:         );
144:         vectorStore.add(documents);
145:         log.info("Successfully loaded {} runbook documents into vector store.", documents. ...
146:
147:     } catch (Exception e) {
148:         log.error("Failed to seed security database schema. Error: {}", e.getMessage(), e) ...
149:     }
150: }
151: }
```

File: backend/src/main/java/com/wsms/controller/AuthenticationController.java

Description: Authentication REST Controller

```
1: package com.wsms.controller;
2:
3: import com.wsms.dto.LoginRequestDto;
4: import com.wsms.dto.RegisterRequestDto;
5: import com.wsms.dto.TokenResponseDto;
6: import com.wsms.service.AuthenticationService;
7: import com.wsms.service.EncryptionService;
8: import jakarta.validation.Valid;
9: import org.springframework.http.ResponseEntity;
10: import org.springframework.web.bind.annotation.*;
11:
12: import java.util.Map;
13:
14: @RestController
15: @RequestMapping("/api/auth")
16: public class AuthenticationController {
17:
18:     private final AuthenticationService authenticationService;
19:     private final EncryptionService encryptionService;
20:
21:     public AuthenticationController(AuthenticationService authenticationService, EncryptionSer ...
22:         this.authenticationService = authenticationService;
23:         this.encryptionService = encryptionService;
24:     }
25:
26:     @GetMapping("/public-key")
27:     public ResponseEntity<Map<String, String>> getPublicKey() {
28:         return ResponseEntity.ok(Map.of("publicKey", encryptionService.getPublicKeyBase64()));
29:     }
30:
31:     @PostMapping("/login")
32:     public ResponseEntity<TokenResponseDto> login(@Valid @RequestBody LoginRequestDto request) ...
33:         return ResponseEntity.ok(authenticationService.login(request));
34:     }
35:
36:     @PostMapping("/register")
37:     public ResponseEntity<String> register(@Valid @RequestBody RegisterRequestDto request) {
38:         authenticationService.register(request);
39:         return ResponseEntity.ok("User registered successfully");
40:     }
41:
42:     @PostMapping("/refresh")
43:     public ResponseEntity<TokenResponseDto> refresh(@RequestBody Map<String, String> request) ...
44:         String refreshToken = request.get("refreshToken");
45:         if (refreshToken == null || refreshToken.isBlank()) {
46:             return ResponseEntity.badRequest().build();
47:         }
48:         return ResponseEntity.ok(authenticationService.refresh(refreshToken));
49:     }
50:
51:     @PostMapping("/logout")
52:     public ResponseEntity<Void> logout(@RequestBody Map<String, String> request) {
53:         String refreshToken = request.get("refreshToken");
54:         if (refreshToken != null && !refreshToken.isBlank()) {
55:             authenticationService.logout(refreshToken);
56:         }
57:         return ResponseEntity.noContent().build();
58:     }
}
```

59: }

File: backend/src/main/java/com/wsms/dto/LoginRequestDto.java

Description: Login Data Transfer Object

```
1: package com.wsms.dto;
2:
3: import jakarta.validation.constraints.NotBlank;
4:
5: public class LoginRequestDto {
6:     @NotBlank(message = "Username or email is required")
7:     private String usernameOrEmail;
8:
9:     @NotBlank(message = "Password is required")
10:    private String password;
11:
12:    private String captchaToken;
13:    private String captchaProvider;
14:
15:    public String getUsernameOrEmail() {
16:        return usernameOrEmail;
17:    }
18:
19:    public void setUsernameOrEmail(String usernameOrEmail) {
20:        this.usernameOrEmail = usernameOrEmail;
21:    }
22:
23:    public String getPassword() {
24:        return password;
25:    }
26:
27:    public void setPassword(String password) {
28:        this.password = password;
29:    }
30:
31:    public String getCaptchaToken() {
32:        return captchaToken;
33:    }
34:
35:    public void setCaptchaToken(String captchaToken) {
36:        this.captchaToken = captchaToken;
37:    }
38:
39:    public String getCaptchaProvider() {
40:        return captchaProvider;
41:    }
42:
43:    public void setCaptchaProvider(String captchaProvider) {
44:        this.captchaProvider = captchaProvider;
45:    }
46: }
```

File: backend/src/main/java/com/wsms/dto/RegisterRequestDto.java

Description: Registration Data Transfer Object

```
1: package com.wsms.dto;
2:
3: import jakarta.validation.constraints.Email;
4: import jakarta.validation.constraints.NotBlank;
5: import jakarta.validation.constraints.Size;
6:
7: public class RegisterRequestDto {
8:     @NotBlank(message = "Username is required")
9:     @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")
10:    private String username;
11:
12:    @NotBlank(message = "Email is required")
13:    @Email(message = "Email must be a valid email address")
14:    @Size(max = 100, message = "Email must be less than 100 characters")
15:    private String email;
16:
17:    @NotBlank(message = "Password is required")
18:    @Size(min = 6, max = 500, message = "Password must be at least 6 characters long")
19:    private String password;
20:
21:    public String getUsername() {
22:        return username;
23:    }
24:
25:    public void setUsername(String username) {
26:        this.username = username;
27:    }
28:
29:    public String getEmail() {
30:        return email;
31:    }
32:
33:    public void setEmail(String email) {
34:        this.email = email;
35:    }
36:
37:    public String getPassword() {
38:        return password;
39:    }
40:
41:    public void setPassword(String password) {
42:        this.password = password;
43:    }
44: }
```

File: backend/src/main/java/com/wsms/service/AuthenticationService.java

Description: Authentication & JWT Generation Service

```
1: package com.wsms.service;
2:
3: import com.wsms.dto.LoginRequestDto;
4: import com.wsms.dto.RegisterRequestDto;
5: import com.wsms.dto.TokenResponseDto;
6: import com.wsms.entity.RefreshToken;
7: import com.wsms.entity.Role;
8: import com.wsms.entity.User;
9: import com.wsms.repository.RefreshTokenRepository;
10: import com.wsms.repository.RoleRepository;
11: import com.wsms.repository.UserRepository;
12: import org.springframework.http.HttpStatus;
13: import org.springframework.security.authentication.AuthenticationManager;
14: import org.springframework.security.authentication.BadCredentialsException;
15: import org.springframework.security.authentication.LockedException;
16: import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
17: import org.springframework.security.core.GrantedAuthority;
18: import org.springframework.security.crypto.password.PasswordEncoder;
19: import org.springframework.stereotype.Service;
20: import org.springframework.transaction.annotation.Transactional;
21: import org.springframework.web.server.ResponseStatusException;
22:
23: import java.time.Instant;
24: import java.time.temporal.ChronoUnit;
25: import java.util.Collections;
26: import java.util.List;
27: import java.util.UUID;
28: import java.util.stream.Collectors;
29:
30: @Service
31: public class AuthenticationService {
32:
33:     private final UserRepository userRepository;
34:     private final RoleRepository roleRepository;
35:     private final RefreshTokenRepository refreshTokenRepository;
36:     private final PasswordEncoder passwordEncoder;
37:     private final JwtService jwtService;
38:     private final AuthenticationManager authenticationManager;
39:     private final RateLimitingService rateLimitingService;
40:     private final RecaptchaService recaptchaService;
41:     private final EncryptionService encryptionService;
42:
43:     public AuthenticationService(
44:         UserRepository userRepository,
45:         RoleRepository roleRepository,
46:         RefreshTokenRepository refreshTokenRepository,
47:         PasswordEncoder passwordEncoder,
48:         JwtService jwtService,
49:         AuthenticationManager authenticationManager,
50:         RateLimitingService rateLimitingService,
51:         RecaptchaService recaptchaService,
52:         EncryptionService encryptionService
53:     ) {
54:         this.userRepository = userRepository;
55:         this.roleRepository = roleRepository;
56:         this.refreshTokenRepository = refreshTokenRepository;
57:         this.passwordEncoder = passwordEncoder;
58:         this.jwtService = jwtService;
```

```
59:         this.authenticationManager = authenticationManager;
60:         this.rateLimitingService = rateLimitingService;
61:         this.recaptchaService = recaptchaService;
62:         this.encryptionService = encryptionService;
63:     }
64:
65:     @Transactional
66:     public TokenResponseDto login(LoginRequestDto request) {
67:         // Perform Captcha Bot Check Verification
68:         recaptchaService.verifyToken(request.getCaptchaToken(), request.getCaptchaProvider());
69:
70:         String usernameOrEmail = request.getUsernameOrEmail();
71:
72:         User user = userRepository.findByUsernameOrEmail(usernameOrEmail, usernameOrEmail)
73:             .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid ...
74:
75:         if (rateLimitingService.isLocked(user)) {
76:             throw new ResponseStatusException(HttpStatus.LOCKED, "Account is locked due to too ...
77:         }
78:
79:         String plainPassword = encryptionService.decrypt(request.getPassword());
80:         try {
81:             authenticationManager.authenticate(
82:                 new UsernamePasswordAuthenticationToken(user.getUsername(), plainPassword)
83:             );
84:         } catch (BadCredentialsException e) {
85:             rateLimitingService.registerFailedAttempt(usernameOrEmail);
86:             User updatedUser = userRepository.findByUsernameOrEmail(usernameOrEmail, usernameOrEmail)
87:                 .orElseThrow(() -> new ResponseStatusException(HttpStatus.LOCKED, "Account has been locked ...
88:             if (rateLimitingService.isLocked(updatedUser)) {
89:                 throw new ResponseStatusException(HttpStatus.LOCKED, "Account has been locked ...
90:             }
91:             throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid username or pa ...
92:         } catch (LockedException e) {
93:             throw new ResponseStatusException(HttpStatus.LOCKED, "Account is locked.");
94:         }
95:
96:         rateLimitingService.resetFailedAttempts(usernameOrEmail);
97:
98:         String accessToken = jwtService.generateAccessToken(user);
99:         RefreshToken refreshToken = createRefreshToken(user);
100:
101:         List<String> roles = user.getAuthorities().stream()
102:             .map(GrantedAuthority::getAuthority)
103:             .collect(Collectors.toList());
104:
105:         return new TokenResponseDto(
106:             accessToken,
107:             refreshToken.getToken(),
108:             user.getUsername(),
109:             user.getEmail(),
110:             roles
111:         );
112:     }
113:
114:     @Transactional
115:     public User register(RegisterRequestDto request) {
116:         if (userRepository.existsByUsername(request.getUsername())) {
117:             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Username is already tak ...
118:         }
119:         if (userRepository.existsByEmail(request.getEmail())) {
120:             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Email is already regist ...
121:         }
122:     }
```

```
122:         String plainPassword = encryptionService.decrypt(request.getPassword());
123:         if (plainPassword.length() < 6) {
124:             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Password must be at lea ...
125:         }
126:
127:         User user = new User(
128:             request.getUsername(),
129:             request.getEmail(),
130:             passwordEncoder.encode(plainPassword)
131:         );
132:
133:         Role userRole = roleRepository.findByName("ROLE_VIEWER")
134:             .orElseThrow(() -> new ResponseStatusException(HttpStatus.INTERNAL_SERVER_ERRO ...
135:
136:         user.setRoles(Collections.singleton(userRole));
137:         return userRepository.save(user);
138:     }
139:
140:     @Transactional
141:     public TokenResponseDto refresh(String token) {
142:         RefreshToken refreshToken = refreshTokenRepository.findByToken(token)
143:             .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Inval ...
144:
145:         if (refreshToken.getExpiryDate().isBefore(Instant.now())) {
146:             refreshTokenRepository.delete(refreshToken);
147:             throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Refresh token expired. ...
148:         }
149:
150:         User user = refreshToken.getUser();
151:         String newAccessToken = jwtService.generateAccessToken(user);
152:
153:         refreshTokenRepository.delete(refreshToken);
154:         RefreshToken newRefreshToken = createRefreshToken(user);
155:
156:         List<String> roles = user.getAuthorities().stream()
157:             .map(GrantedAuthority::getAuthority)
158:             .collect(Collectors.toList());
159:
160:         return new TokenResponseDto(
161:             newAccessToken,
162:             newRefreshToken.getToken(),
163:             user.getUsername(),
164:             user.getEmail(),
165:             roles
166:         );
167:     }
168:
169:     @Transactional
170:     public void logout(String token) {
171:         refreshTokenRepository.findByToken(token).ifPresent(refreshTokenRepository::delete);
172:     }
173:
174:     private RefreshToken createRefreshToken(User user) {
175:         refreshTokenRepository.deleteByUser(user);
176:
177:         RefreshToken refreshToken = new RefreshToken();
178:         refreshToken.setUser(user);
179:         refreshToken.setExpiryDate(Instant.now().plus(7, ChronoUnit.DAYS));
180:         refreshToken.setToken(UUID.randomUUID().toString());
181:         return refreshTokenRepository.save(refreshToken);
182:     }
183: }
```

File: backend/src/main/java/com/wsms/service/EncryptionService.java

Description: E2E RSA Password Decryption Service

```
1: package com.wsms.service;
2:
3: import org.springframework.stereotype.Service;
4:
5: import javax.crypto.Cipher;
6: import javax.crypto.spec.OAEPParameterSpec;
7: import javax.crypto.spec.PSource;
8: import java.nio.charset.StandardCharsets;
9: import java.security.KeyPair;
10: import java.security.KeyPairGenerator;
11: import java.security.spec.MGF1ParameterSpec;
12: import java.util.Base64;
13:
14: @Service
15: public class EncryptionService {
16:
17:     private final KeyPair keyPair;
18:
19:     public EncryptionService() {
20:         try {
21:             KeyPairGenerator generator = KeyPairGenerator.getInstance("RSA");
22:             generator.initialize(2048);
23:             this.keyPair = generator.generateKeyPair();
24:         } catch (Exception e) {
25:             throw new IllegalStateException("Failed to initialize RSA KeyPairGenerator", e);
26:         }
27:     }
28:
29:     public String getPublicKeyBase64() {
30:         return Base64.getEncoder().encodeToString(keyPair.getPublic().getEncoded());
31:     }
32:
33:     public String decrypt(String encryptedBase64) {
34:         try {
35:             byte[] encryptedBytes = Base64.getDecoder().decode(encryptedBase64.trim());
36:             Cipher cipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
37:             OAEPParameterSpec oaepSpec = new OAEPParameterSpec(
38:                 "SHA-256",
39:                 "MGF1",
40:                 MGF1ParameterSpec.SHA256,
41:                 PSource.PSpecified.DEFAULT
42:             );
43:             cipher.init(Cipher.DECRYPT_MODE, keyPair.getPrivate(), oaepSpec);
44:             byte[] decryptedBytes = cipher.doFinal(encryptedBytes);
45:             return new String(decryptedBytes, StandardCharsets.UTF_8);
46:         } catch (Exception e) {
47:             throw new IllegalArgumentException("Failed to decrypt password. Make sure it was e ...");
48:         }
49:     }
50: }
```

File: backend/src/main/java/com/wsms/service/RecaptchaService.java

Description: reCAPTCHA & Turnstile Validator

```
1: package com.wsms.service;
2:
3: import com.wsms.dto.RecaptchaResponse;
4: import org.slf4j.Logger;
5: import org.slf4j.LoggerFactory;
6: import org.springframework.beans.factory.annotation.Value;
7: import org.springframework.http.*;
8: import org.springframework.stereotype.Service;
9: import org.springframework.util.LinkedMultiValueMap;
10: import org.springframework.util.MultiValueMap;
11: import org.springframework.web.client.RestTemplate;
12: import org.springframework.web.server.ResponseStatusException;
13:
14: @Service
15: public class RecaptchaService {
16:     private static final Logger log = LoggerFactory.getLogger(RecaptchaService.class);
17:
18:     private final RestTemplate restTemplate = new RestTemplate();
19:
20:     @Value("${google.recaptcha.secret}")
21:     private String recaptchaSecret;
22:
23:     @Value("${google.recaptcha.threshold:0.5}")
24:     private double recaptchaThreshold;
25:
26:     @Value("${cloudflare.turnstile.secret}")
27:     private String turnstileSecret;
28:
29:     private static final String GOOGLE_VERIFY_URL = "https://www.google.com/recaptcha/api/site ...
30:     private static final String CLOUDFLARE_VERIFY_URL = "https://challenges.cloudflare.com/tur ...
31:
32:     public void verifyToken(String token, String provider) {
33:         if (token == null || token.isBlank()) {
34:             log.warn("Missing captcha token");
35:             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Missing captcha token v ...
36:         }
37:
38:         String verifyUrl;
39:         String secret;
40:         if ("turnstile".equalsIgnoreCase(provider)) {
41:             verifyUrl = CLOUDFLARE_VERIFY_URL;
42:             secret = turnstileSecret;
43:             log.debug("Verifying token with Cloudflare Turnstile");
44:         } else {
45:             verifyUrl = GOOGLE_VERIFY_URL;
46:             secret = recaptchaSecret;
47:             log.debug("Verifying token with Google reCAPTCHA v3");
48:         }
49:
50:         HttpHeaders headers = new HttpHeaders();
51:         headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
52:
53:         MultiValueMap<String, String> map = new LinkedMultiValueMap<>();
54:         map.add("secret", secret);
55:         map.add("response", token);
56:
57:         HttpEntity<MultiValueMap<String, String>> request = new HttpEntity<>(map, headers);
58:     }
```

```
59:         try {
60:             ResponseEntity<RecaptchaResponse> responseEntity = restTemplate.postForEntity(veri ...
61:             RecaptchaResponse body = responseEntity.getBody();
62:
63:             if (body == null || !body.isSuccess()) {
64:                 String errors = (body != null && body.getErrorCodes() != null) ? String.join(" ...
65:                 log.warn("Captcha verification failed. Provider: {}, Error codes: {}", provide ...
66:                 throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Bot check verificat ...
67:             }
68:
69:             // Google reCAPTCHA v3 risk score validation
70:             if (!"turnstile".equalsIgnoreCase(provider)) {
71:                 log.debug("reCAPTCHA v3 score: {}", body.getScore());
72:                 if (body.getScore() < recaptchaThreshold) {
73:                     log.warn("reCAPTCHA risk score {} is below threshold {}", body.getScore(), ...
74:                     throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Security threat ...
75:                 }
76:             }
77:
78:             log.info("Captcha verification succeeded for provider: {}", provider);
79:         } catch (ResponseStatusException e) {
80:             throw e;
81:         } catch (Exception e) {
82:             log.error("Error connecting to captcha verification API: {}", e.getMessage(), e);
83:             throw new ResponseStatusException(HttpStatus.INTERNAL_SERVER_ERROR, "Captcha verif ...
84:         }
85:     }
86: }
```

File: backend/src/main/java/com/wsms/service/RagService.java*Description: Semantic Vector Search Playbooks Retriever*

```

1: package com.wsms.service;
2:
3: import org.springframework.ai.document.Document;
4: import org.springframework.ai.embedding.EmbeddingClient;
5: import org.springframework.ai.vectorstore.SearchRequest;
6: import org.springframework.ai.vectorstore.SimpleVectorStore;
7: import org.springframework.jdbc.core.JdbcTemplate;
8: import org.springframework.stereotype.Service;
9: import java.util.List;
10: import java.util.Map;
11: import java.util.stream.Collectors;
12:
13: @Service
14: public class RagService {
15:
16:     private final JdbcTemplate jdbcTemplate;
17:     private final SimpleVectorStore vectorStore;
18:     private final EmbeddingClient embeddingClient;
19:
20:     public RagService(JdbcTemplate jdbcTemplate, SimpleVectorStore vectorStore, EmbeddingClient embeddingClient) {
21:         this.jdbcTemplate = jdbcTemplate;
22:         this.vectorStore = vectorStore;
23:         this.embeddingClient = embeddingClient;
24:     }
25:
26:     public String getDatabaseSchema(String query) {
27:         String websitesSchema = "TABLE websites (\n" +
28:             "    id SERIAL PRIMARY KEY,\n" +
29:             "    website_name VARCHAR(100) NOT NULL,\n" +
30:             "    website_url VARCHAR(255) NOT NULL UNIQUE,\n" +
31:             "    check_interval INTEGER DEFAULT 60,\n" +
32:             "    status VARCHAR(20) DEFAULT 'PENDING',\n" +
33:             "    response_time DOUBLE PRECISION,\n" +
34:             "    ewma_response_time DOUBLE PRECISION,\n" +
35:             "    ssl_expiry_date TIMESTAMP,\n" +
36:             "    dns_lookup_time DOUBLE PRECISION,\n" +
37:             "    ssl_issuer VARCHAR(255),\n" +
38:             "    protocol VARCHAR(10),\n" +
39:             "    ssl_expiry_threshold INTEGER DEFAULT 30\n" +
40:             ");\n";
41:
42:         if (query == null || query.trim().isEmpty()) {
43:             return websitesSchema;
44:         }
45:
46:         String lowerQuery = query.toLowerCase();
47:         boolean needsLogs = lowerQuery.contains("history") || lowerQuery.contains("logs") ||
48:             lowerQuery.contains("trend") || lowerQuery.contains("past") ||
49:             lowerQuery.contains("change") || lowerQuery.contains("previous") ||
50:             lowerQuery.contains("checked") || lowerQuery.contains("record");
51:
52:         if (needsLogs) {
53:             return websitesSchema +
54:                 "TABLE monitoring_logs (\n" +
55:                 "    id BIGSERIAL PRIMARY KEY,\n" +
56:                 "    website_id INTEGER REFERENCES websites(id),\n" +
57:                 "    status_code INTEGER,\n" +
58:                 "    response_time INTEGER,\n" +

```

```

59:         " status VARCHAR(10),\n" +
60:         " checked_at TIMESTAMP NOT NULL\n" +
61:         ");";
62:     }
63:
64:     return websitesSchema + "-- Note: Do not query or join any other tables. ONLY query th ...
65: }
66:
67: public List<Map<String, Object>> executeGeneratedQuery(String sql) {
68:     if (sql == null || sql.trim().isEmpty()) {
69:         return null;
70:     }
71:
72:     String cleanSql = sql.trim().toLowerCase();
73:
74:     // Safety check: only allow SELECT queries to prevent database corruption/modification
75:     if (!cleanSql.startsWith("select")) {
76:         throw new IllegalArgumentException("For safety, only SELECT queries are permitted. ...
77:     }
78:
79:     // Remove dangerous keywords to prevent multi-statements or privilege escalation
80:     if (cleanSql.contains("insert") || cleanSql.contains("update") ||
81:         cleanSql.contains("delete") || cleanSql.contains("drop") ||
82:         cleanSql.contains("alter") || cleanSql.contains("truncate") ||
83:         cleanSql.contains("grant") || cleanSql.contains("revoke")) {
84:         throw new IllegalArgumentException("Query contains unauthorized keywords (INSERT, ...
85:     }
86:
87:     // Limit results to prevent memory exhaustion
88:     if (!cleanSql.contains("limit")) {
89:         sql = sql.trim() + " LIMIT 50";
90:     }
91:
92:     return jdbcTemplate.queryForList(sql);
93: }
94:
95: public String searchRunbooks(String query) {
96:     if (query == null || query.trim().isEmpty()) {
97:         return "";
98:     }
99:
100:    try {
101:        List<Double> queryEmbedding = embeddingClient.embed(query);
102:        List<Document> similarDocuments = vectorStore.similaritySearch(
103:            SearchRequest.query(query).withTopK(5)
104:        );
105:
106:        System.out.println("DEBUG RAG SEARCH FOR: \"" + query + "\" (total retrieved: " + ...
107:
108:        return similarDocuments.stream()
109:            .filter(doc -> {
110:                List<Double> docEmbedding = doc.getEmbedding();
111:                double similarity = cosineSimilarity(queryEmbedding, docEmbedding);
112:                System.out.println(" - Title: " + doc.getMetadata().get("title") + " | Cu ...
113:                // Filter: only keep documents with similarity >= 0.35
114:                return similarity >= 0.35;
115:            })
116:            .map(Document::getContent)
117:            .collect(Collectors.joining("\n\n"));
118:
119:    } catch (Exception e) {
120:        System.err.println("Error performing similarity search: " + e.getMessage());
121:        // Fallback to keyword search if vector search fails

```

```
122:         String sql = "SELECT content FROM security_runbooks WHERE title ILIKE ? OR content ...
123:         String wildCard = "%" + query.trim() + "%";
124:         List<String> results = jdbcTemplate.query(sql,
125:             (rs, rowNum) -> rs.getString("content"),
126:             wildCard, wildCard);
127:
128:         return String.join("\n\n", results);
129:     }
130: }
131:
132: private double cosineSimilarity(List<Double> v1, List<Double> v2) {
133:     if (v1 == null || v2 == null || v1.size() != v2.size() || v1.isEmpty()) {
134:         return 0.0;
135:     }
136:     double dotProduct = 0.0;
137:     double norm1 = 0.0;
138:     double norm2 = 0.0;
139:     for (int i = 0; i < v1.size(); i++) {
140:         double x = v1.get(i);
141:         double y = v2.get(i);
142:         dotProduct += x * y;
143:         norm1 += x * x;
144:         norm2 += y * y;
145:     }
146:     if (norm1 == 0.0 || norm2 == 0.0) {
147:         return 0.0;
148:     }
149:     return dotProduct / (Math.sqrt(norm1) * Math.sqrt(norm2));
150: }
151: }
152:
```

File: backend/src/main/java/com/wsms/service/AiService.java

Description: Ollama LLM Prompts & Observability Assistant

```
1: package com.wsms.service;
2:
3: import org.springframework.ai.chat.ChatResponse;
4: import org.springframework.ai.chat.prompt.Prompt;
5: import org.springframework.ai.chat.prompt.PromptTemplate;
6: import org.springframework.ai.ollama.OllamaChatClient;
7: import org.springframework.ai.ollama.api.OllamaOptions;
8: import org.springframework.stereotype.Service;
9: import java.util.List;
10: import java.util.Map;
11: import java.util.regex.Matcher;
12: import java.util.regex.Pattern;
13:
14: @Service
15: public class AiService {
16:
17:     private final OllamaChatClient chatClient;
18:     private final RagService ragService;
19:
20:     public AiService(OllamaChatClient chatClient, RagService ragService) {
21:         this.chatClient = chatClient;
22:         this.ragService = ragService;
23:     }
24:
25:     public String generateSql(String userInput) {
26:         if (userInput == null || userInput.trim().isEmpty()) {
27:             return "";
28:         }
29:
30:         // Only query database if user specifically asks for database facts or lists
31:         String lowerInput = userInput.toLowerCase();
32:         if (!lowerInput.contains("show") && !lowerInput.contains("select") &&
33:             !lowerInput.contains("get") && !lowerInput.contains("website") &&
34:             !lowerInput.contains("log") && !lowerInput.contains("outage") &&
35:             !lowerInput.contains("uptime") && !lowerInput.contains("down") &&
36:             !lowerInput.contains("up") && !lowerInput.contains("latency") &&
37:             !lowerInput.contains("spike")) {
38:             return "";
39:         }
40:
41:         String promptTemplate =
42:             "Translate the natural language request to a single, read-only PostgreSQL SELECT q ...
43:             "PostgreSQL Schema:\n" +
44:             "{schema}\n\n" +
45:             "Request: \"{query}\" \n\n" +
46:             "Rules:\n" +
47:             "1. Output format: [SQL] <query> [/SQL]\n" +
48:             "2. Do NOT write markdown code blocks (``sql or ``) inside or outside the SQL ta ...
49:             "3. Do NOT include semi-colons inside the SQL tags.\n" +
50:             "4. Start directly with the [SQL] tag. Do not write any conversational text.\n" +
51:             "5. Select only specific necessary columns (e.g. website_name, status, response_ti ...
52:
53:         PromptTemplate template = new PromptTemplate(promptTemplate);
54:         OllamaOptions options = OllamaOptions.create()
55:             .withTemperature(0.0f)
56:             .withNumPredict(80);
57:
58:         Prompt prompt = new Prompt(template.createMessage(Map.of(
```

```

59:         "schema", ragService.getDatabaseSchema(userInput),
60:         "query", userInput
61:     )), options);
62:
63:     try {
64:         ChatResponse response = chatClient.call(prompt);
65:         String text = response.getResult().getOutput().getContent();
66:         System.out.println("DEBUG RAW LLM SQL OUTPUT: " + text);
67:
68:         Pattern pattern = Pattern.compile("\\[SQL\\](.*?)?(?:\\[/SQL\\]|$)", Pattern.CASE_I ...
69:         Matcher matcher = pattern.matcher(text);
70:         String sql = "";
71:         if (matcher.find()) {
72:             sql = matcher.group(1).trim();
73:         } else {
74:             // Fallback: search for direct select query if tags were omitted by the model
75:             String lowerText = text.toLowerCase();
76:             int selectIdx = lowerText.indexOf("select");
77:             if (selectIdx != -1) {
78:                 sql = text.substring(selectIdx).trim();
79:                 int semiColonIdx = sql.indexOf(";");
80:                 if (semiColonIdx != -1) {
81:                     sql = sql.substring(0, semiColonIdx).trim();
82:                 }
83:                 int newlineIdx = sql.indexOf("\n");
84:                 if (newlineIdx != -1) {
85:                     sql = sql.substring(0, newlineIdx).trim();
86:                 }
87:             }
88:         }
89:
90:         if (sql != null && !sql.isEmpty()) {
91:             // Clean up any remaining formatting markdown code blocks
92:             sql = sql.replace("`sql", "").replace("`", "").trim();
93:             // Ensure query ends without semi-colon for Spring JdbcTemplate handling
94:             if (sql.endsWith(";")) {
95:                 sql = sql.substring(0, sql.length() - 1);
96:             }
97:             sql = sql.replace("`", "").trim();
98:             return sql;
99:         }
100:     } catch (Exception e) {
101:         // Log LLM communication exception and return empty so fallback handles it
102:         System.err.println("Error calling Ollama to generate SQL: " + e.getMessage());
103:     }
104:     return "";
105: }
106:
107: public String analyzeTelemetryAndAnswer(String userInput, String sql, List<Map<String, Obj ...
108:     String runbooks = ragService.searchRunbooks(userInput);
109:
110:     String promptTemplate =
111:         "You are the WSMS Observability Assistant, a helpful AI assistant built into the W ...
112:         "User request: \"{query}\"\\n" +
113:         "SQL query executed: {sql}\\n" +
114:         "Data returned:\\n{results}\\n\\n" +
115:         "Runbook advice:\\n{runbooks}\\n\\n" +
116:         "Instructions:\\n" +
117:         "1. If the user's request is a general question, greeting, or system access questi ...
118:         "2. If the user is asking about site status, outages, latency, or database details ...
119:         "3. Keep the response concise, helpful, and formatted in clean markdown.";
120:
121:     PromptTemplate template = new PromptTemplate(promptTemplate);

```

```
122:         OllamaOptions options = OllamaOptions.create()
123:             .withTemperature(0.1f)
124:             .withNumPredict(150);
125:
126:         Prompt prompt = new Prompt(template.createMessage(Map.of(
127:             "query", userInput,
128:             "sql", sql == null || sql.isEmpty() ? "None" : sql,
129:             "results", formatQueryResultsCompact(queryResults),
130:             "runbooks", runbooks == null || runbooks.isEmpty() ? "No specific runbook match fo ...
131:         )), options);
132:
133:         try {
134:             ChatResponse response = chatClient.call(prompt);
135:             return response.getResult().getOutput().getContent();
136:         } catch (Exception e) {
137:             return "The AI Monitoring Assistant is currently unavailable because the local Oll ...
138:         }
139:     }
140:
141:     private String formatQueryResultsCompact(List<Map<String, Object>> results) {
142:         if (results == null || results.isEmpty()) {
143:             return "No database records retrieved.";
144:         }
145:
146:         StringBuilder sb = new StringBuilder();
147:         // Extract headers from first row
148:         Map<String, Object> firstRow = results.get(0);
149:         sb.append(String.join(", ", firstRow.keySet())).append("\n");
150:
151:         int limit = Math.min(results.size(), 5);
152:         for (int i = 0; i < limit; i++) {
153:             Map<String, Object> row = results.get(i);
154:             List<String> values = new java.util.ArrayList<>();
155:             for (Object val : row.values()) {
156:                 values.add(val == null ? "null" : val.toString());
157:             }
158:             sb.append(String.join(", ", values)).append("\n");
159:         }
160:
161:         if (results.size() > 5) {
162:             sb.append("... and ").append(results.size() - 5).append(" more records exist.\n");
163:         }
164:
165:         return sb.toString();
166:     }
167: }
```

File: frontend/src/views/Login.vue

Description: Vue.js E2E Encrypted Auth Component

```
1: <script setup>
2: import { ref, onMounted, onUnmounted, watch, nextTick } from 'vue';
3: import { useRouter } from 'vue-router';
4: import apiClient from '../api/client';
5: import { useAuthStore } from '../store/auth';
6: import * as THREE from 'three';
7:
8: const showTurnstile = ref(true);
9: const turnstileToken = ref('');
10:
11: const RECAPTCHA_SITE_KEY = '6LeIxActAAAAAJcZVRqyHh71UMTerE_Yth5aqaF1';
12: const TURNSTILE_SITE_KEY = '1x00000000000000000000AA';
13:
14: const router = useRouter();
15: const authStore = useAuthStore();
16:
17: // Form state
18: const usernameOrEmail = ref('');
19: const email = ref('');
20: const password = ref('');
21: const confirmPassword = ref('');
22: const loading = ref(false);
23: const errorMessage = ref('');
24: const successMessage = ref('');
25: const isShaking = ref(false);
26: const isRegisterMode = ref(false);
27:
28: const publicKeyPEM = ref('');
29:
30: const importPublicKey = async (pemBase64) => {
31:   const binaryDerString = window.atob(pemBase64);
32:   const binaryDer = new ArrayBuffer(binaryDerString.length);
33:   const binaryDerView = new Uint8Array(binaryDer);
34:   for (let i = 0; i < binaryDerString.length; i++) {
35:     binaryDerView[i] = binaryDerString.charCodeAt(i);
36:   }
37:   return await window.crypto.subtle.importKey(
38:     "spki",
39:     binaryDer,
40:     {
41:       name: "RSA-OAEP",
42:       hash: "SHA-256"
43:     },
44:     true,
45:     ["encrypt"]
46:   );
47: };
48:
49: const encryptPassword = async (plainText) => {
50:   if (!publicKeyPEM.value) {
51:     throw new Error('Encryption key not loaded. Please try again.');
```

```
59:     },
60:     importedKey,
61:     data
62:   );
63:   const encryptedBytes = new Uint8Array(encryptedBuffer);
64:   let binary = '';
65:   const len = encryptedBytes.byteLength;
66:   for (let i = 0; i < len; i++) {
67:     binary += String.fromCharCode(encryptedBytes[i]);
68:   }
69:   return window.btoa(binary);
70: };
71:
72: const fetchPublicKey = async () => {
73:   try {
74:     const response = await apiClient.get('/api/auth/public-key');
75:     publicKeyPEM.value = response.data.publicKey;
76:   } catch (err) {
77:     console.error('Failed to fetch public key for E2E password encryption:', err);
78:     errorMessage.value = 'Failed to establish secure connection parameters. Please refresh the ...
79:   }
80: };
81:
82: // Three.js container ref
83: const canvasContainer = ref(null);
84: let scene, camera, renderer, particleSystem, linesSystem, starsSystem;
85: let animationFrameId;
86:
87: // Voice alert helper
88: const speakAlert = (text) => {
89:   if (typeof window === 'undefined' || !('speechSynthesis' in window)) return;
90:   window.speechSynthesis.cancel();
91:   const utterance = new SpeechSynthesisUtterance(text);
92:
93:   const voices = window.speechSynthesis.getVoices();
94:   let femaleVoice = voices.find(voice => {
95:     const name = voice.name.toLowerCase();
96:     const lang = voice.lang.toLowerCase();
97:     if (!lang.startsWith('en')) return false;
98:     return name.includes('zira') ||
99:        name.includes('samantha') ||
100:        name.includes('siri') ||
101:        name.includes('google us english') ||
102:        name.includes('natural') && name.includes('female') ||
103:        name.includes('female') ||
104:        name.includes('aria') ||
105:        name.includes('hazel') ||
106:        name.includes('Susan');
107:   });
108:
109:   if (!femaleVoice) {
110:     femaleVoice = voices.find(voice => {
111:       const name = voice.name.toLowerCase();
112:       const lang = voice.lang.toLowerCase();
113:       if (!lang.startsWith('en')) return false;
114:       return !name.includes('david') &&
115:          !name.includes('mark') &&
116:          !name.includes('ravi') &&
117:          !name.includes('george') &&
118:          !name.includes('male');
119:     });
120:   }
121: }
```

```
122:   if (femaleVoice) utterance.voice = femaleVoice;
123:   utterance.rate = 0.92;
124:   utterance.pitch = 1.15;
125:   window.speechSynthesis.speak(utterance);
126: };
127:
128: // Handle submission
129: const handleSubmit = async () => {
130:   if (loading.value) return;
131:   errorMessage.value = '';
132:   successMessage.value = '';
133:   isShaking.value = false;
134:
135:   if (isRegisterMode.value) {
136:     if (password.value !== confirmPassword.value) {
137:       errorMessage.value = 'Passwords do not match.';
138:       isShaking.value = true;
139:       speakAlert('Passwords do not match.');
```

140: setTimeout(() => { isShaking.value = false; }, 600);

141: return;

142: }

143:

144: loading.value = true;

145: try {

146: if (!publicKeyPEM.value) {

147: await fetchPublicKey();

148: }

149: const encryptedPassword = await encryptPassword(password.value);

150: await apiClient.post('/api/auth/register', {

151: username: usernameOrEmail.value,

152: email: email.value,

153: password: encryptedPassword

154: });

155:

156: speakAlert('Account created successfully. You can now log in.');

157: successMessage.value = 'Registration successful! Please log in below.';

158: isRegisterMode.value = false;

159: password.value = '';

160: confirmPassword.value = '';

161: } catch (error) {

162: errorMessage.value = error.response?.data?.message || 'Registration failed. Please check ...

163: isShaking.value = true;

164: speakAlert('Registration failed.');

165: setTimeout(() => { isShaking.value = false; }, 600);

166: } finally {

167: loading.value = false;

168: }

169: } else {

170: // Verify that the Turnstile captcha token is present

171: if (!turnstileToken.value) {

172: errorMessage.value = 'Please complete the CAPTCHA verification check.';

173: isShaking.value = true;

174: speakAlert('Please complete the verification.');

175: setTimeout(() => { isShaking.value = false; }, 600);

176: return;

177: }

178:

179: loading.value = true;

180: try {

181: if (!publicKeyPEM.value) {

182: await fetchPublicKey();

183: }

184: const encryptedPassword = await encryptPassword(password.value);

```
185:     const response = await apiClient.post('/api/auth/login', {
186:       usernameOrEmail: usernameOrEmail.value,
187:       password: encryptedPassword,
188:       captchaToken: turnstileToken.value,
189:       captchaProvider: 'turnstile'
190:     });
191:
192:     authStore.setTokens(response.data);
193:
194:     const role = response.data.roles && response.data.roles.length > 0 ? response.data.roles ...
195:     speakAlert('Access granted. Welcome back, ${response.data.username}. Initiating monitori ...
196:
197:     router.push('/');
198:   } catch (error) {
199:     errorMessage.value = error.response?.data?.message || 'Authentication failed. Please ver ...
200:     isShaking.value = true;
201:
202:     if (window.turnstile) {
203:       window.turnstile.reset();
204:       turnstileToken.value = '';
205:     }
206:
207:     if (error.response?.status === 423 || error.response?.status === 401 && errorMessage.val ...
208:       speakAlert("Access denied. Account is temporarily locked due to excessive failed attem ...
209:     } else {
210:       speakAlert("Access denied. Invalid credentials.");
211:     }
212:
213:     setTimeout(() => {
214:       isShaking.value = false;
215:     }, 600);
216:   } finally {
217:     loading.value = false;
218:   }
219: }
220: };
221:
222: const handleGoogleStub = () => {
223:   speakAlert("Google integration is configured in client workflow stub mode. Please use standa ...
224:   errorMessage.value = 'Google login is stubbed. Please use standard credentials.';
225:   isShaking.value = true;
226:   setTimeout(() => { isShaking.value = false; }, 600);
227: };
228:
229: const handleGithubStub = () => {
230:   speakAlert("GitHub authentication is configured in client workflow sandbox mode. Please use ...
231:   errorMessage.value = 'GitHub login is stubbed. Please use standard credentials.';
232:   isShaking.value = true;
233:   setTimeout(() => { isShaking.value = false; }, 600);
234: };
235:
236: let mouseX = 0, mouseY = 0;
237:
238: // Initialize Three.js 3D Visualizer
239: const initThree = () => {
240:   if (!canvasContainer.value) return;
241:
242:   let width = canvasContainer.value.clientWidth;
243:   let height = canvasContainer.value.clientHeight;
244:   if (width === 0 || height === 0) {
245:     const rect = canvasContainer.value.getBoundingClientRect();
246:     width = rect.width || 600;
247:     height = rect.height || 600;
```

```
248:   }
249:
250:   // Scene
251:   scene = new THREE.Scene();
252:   scene.fog = new THREE.FogExp2(0x060913, 0.008);
253:
254:   // Camera
255:   camera = new THREE.PerspectiveCamera(55, width / height, 0.1, 1000);
256:   camera.position.z = 85;
257:
258:   // Renderer
259:   renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
260:   renderer.setSize(width, height);
261:   renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
262:
263:   renderer.domElement.style.width = '100%';
264:   renderer.domElement.style.height = '100%';
265:   renderer.domElement.style.position = 'absolute';
266:   renderer.domElement.style.top = '0';
267:   renderer.domElement.style.left = '0';
268:
269:   canvasContainer.value.appendChild(renderer.domElement);
270:
271:   // 1. Digital Holographic Globe Core
272:   const globeGroup = new THREE.Group();
273:
274:   // Wireframe globe mesh
275:   const sphereGeo = new THREE.SphereGeometry(18, 22, 22);
276:   const wireframeMat = new THREE.MeshBasicMaterial({
277:     color: 0x4f46e5,
278:     wireframe: true,
279:     transparent: true,
280:     opacity: 0.14,
281:     blending: THREE.AdditiveBlending
282:   });
283:   const wireframeGlobe = new THREE.Mesh(sphereGeo, wireframeMat);
284:   globeGroup.add(wireframeGlobe);
285:
286:   // Globe point cloud (dense surface nodes representing monitoring points)
287:   const globePointsGeo = new THREE.SphereGeometry(18.1, 36, 36);
288:   const globePointsMat = new THREE.PointsMaterial({
289:     size: 0.6,
290:     color: 0x06b6d4, // Neon Cyan
291:     transparent: true,
292:     opacity: 0.5,
293:     blending: THREE.AdditiveBlending
294:   });
295:   const globePointCloud = new THREE.Points(globePointsGeo, globePointsMat);
296:   globeGroup.add(globePointCloud);
297:
298:   scene.add(globeGroup);
299:
300:   // 2. Cybernetic Monitoring Orbiting Rings & Data Packets
301:   const ringsGroup = new THREE.Group();
302:   const ringsCount = 3;
303:   const ringsData = [];
304:
305:   const createCircleTexture = (colorStr, size = 16) => {
306:     const canvas = document.createElement('canvas');
307:     canvas.width = size;
308:     canvas.height = size;
309:     const ctx = canvas.getContext('2d');
310:     const grad = ctx.createRadialGradient(size/2, size/2, 0, size/2, size/2, size/2);
```

```
311:     grad.addColorStop(0, colorStr);
312:     grad.addColorStop(1, 'rgba(0,0,0,0)');
313:     ctx.fillStyle = grad;
314:     ctx.fillRect(0, 0, size, size);
315:     return new THREE.CanvasTexture(canvas);
316: };
317:
318: const ringColors = [0x8b5cf6, 0x10b981, 0x3b82f6]; // Violet, Emerald, Blue
319:
320: for (let r = 0; r < ringsCount; r++) {
321:     const ringRadius = 23 + r * 3;
322:     const ringPointsCount = 60;
323:     const ringGeometry = new THREE.BufferGeometry();
324:     const ringPositions = new Float32Array(ringPointsCount * 3);
325:
326:     for (let i = 0; i < ringPointsCount; i++) {
327:         const angle = (i / ringPointsCount) * Math.PI * 2;
328:         ringPositions[i * 3] = Math.cos(angle) * ringRadius;
329:         ringPositions[i * 3 + 1] = Math.sin(angle) * ringRadius;
330:         ringPositions[i * 3 + 2] = 0;
331:     }
332:
333:     ringGeometry.setAttribute('position', new THREE.BufferAttribute(ringPositions, 3));
334:
335:     const ringMaterial = new THREE.PointsMaterial({
336:         size: 2.2,
337:         color: ringColors[r],
338:         map: createCircleTexture('rgba(255,255,255,1)'),
339:         transparent: true,
340:         opacity: 0.85,
341:         blending: THREE.AdditiveBlending,
342:         depthWrite: false
343:     });
344:
345:     const ringPoints = new THREE.Points(ringGeometry, ringMaterial);
346:
347:     // Inclines rings randomly
348:     ringPoints.rotation.x = Math.random() * Math.PI;
349:     ringPoints.rotation.y = Math.random() * Math.PI;
350:
351:     ringsGroup.add(ringPoints);
352:     ringsData.push({
353:         mesh: ringPoints,
354:         speedX: 0.0025 + r * 0.0015,
355:         speedY: 0.004 - r * 0.0015,
356:         speedZ: 0.006 + r * 0.001
357:     });
358: }
359: scene.add(ringsGroup);
360:
361: // 3. Constellation Nodes & Connecting Telemetry Lines
362: const particlesCount = 100;
363: const positions = new Float32Array(particlesCount * 3);
364: const colors = new Float32Array(particlesCount * 3);
365:
366: const colorPalette = [
367:     new THREE.Color(0x8b5cf6), // Violet
368:     new THREE.Color(0x6366f1), // Indigo
369:     new THREE.Color(0x10b981) // Emerald
370: ];
371:
372: for (let i = 0; i < particlesCount; i++) {
373:     // Distribute nodes in a sphere shell around the globe
```

```
374:     const u = Math.random();
375:     const v = Math.random();
376:     const theta = u * 2.0 * Math.PI;
377:     const phi = Math.acos(2.0 * v - 1.0);
378:     const r = 32 + Math.random() * 12;
379:
380:     positions[i * 3] = r * Math.sin(phi) * Math.cos(theta);
381:     positions[i * 3 + 1] = r * Math.sin(phi) * Math.sin(theta);
382:     positions[i * 3 + 2] = r * Math.cos(phi);
383:
384:     const color = colorPalette[Math.floor(Math.random() * colorPalette.length)];
385:     colors[i * 3] = color.r;
386:     colors[i * 3 + 1] = color.g;
387:     colors[i * 3 + 2] = color.b;
388: }
389:
390: const particleGeometry = new THREE.BufferGeometry();
391: particleGeometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
392: particleGeometry.setAttribute('color', new THREE.BufferAttribute(colors, 3));
393:
394: const particleMaterial = new THREE.PointsMaterial({
395:     size: 2.8,
396:     map: createCircleTexture('rgba(255,255,255,1)'),
397:     vertexColors: true,
398:     transparent: true,
399:     opacity: 0.8,
400:     blending: THREE.AdditiveBlending,
401:     depthWrite: false
402: });
403:
404: particleSystem = new THREE.Points(particleGeometry, particleMaterial);
405: scene.add(particleSystem);
406:
407: // 4. Interconnecting Lines
408: const linesGeometry = new THREE.BufferGeometry();
409: const linePositions = [];
410: const maxConnections = 80;
411:
412: for (let i = 0; i < particlesCount; i++) {
413:     for (let j = i + 1; j < particlesCount; j++) {
414:         const dx = positions[i * 3] - positions[j * 3];
415:         const dy = positions[i * 3 + 1] - positions[j * 3 + 1];
416:         const dz = positions[i * 3 + 2] - positions[j * 3 + 2];
417:         const dist = Math.sqrt(dx * dx + dy * dy + dz * dz);
418:
419:         if (dist < 15 && linePositions.length / 6 < maxConnections) {
420:             linePositions.push(positions[i * 3], positions[i * 3 + 1], positions[i * 3 + 2]);
421:             linePositions.push(positions[j * 3], positions[j * 3 + 1], positions[j * 3 + 2]);
422:         }
423:     }
424: }
425:
426: linesGeometry.setAttribute('position', new THREE.BufferAttribute(new Float32Array(linePositi ...
427: const linesMaterial = new THREE.LineBasicMaterial({
428:     color: 0x6366f1,
429:     transparent: true,
430:     opacity: 0.12,
431:     blending: THREE.AdditiveBlending
432: });
433:
434: linesSystem = new THREE.LineSegments(linesGeometry, linesMaterial);
435: scene.add(linesSystem);
436:
```

```
437: // 5. Background Stars
438: const starsCount = 300;
439: const starsPositions = new Float32Array(starsCount * 3);
440: for (let i = 0; i < starsCount; i++) {
441:     starsPositions[i * 3] = (Math.random() - 0.5) * 400;
442:     starsPositions[i * 3 + 1] = (Math.random() - 0.5) * 400;
443:     starsPositions[i * 3 + 2] = (Math.random() - 0.5) * 400;
444: }
445: const starsGeometry = new THREE.BufferGeometry();
446: starsGeometry.setAttribute('position', new THREE.BufferAttribute(starsPositions, 3));
447: const starsMaterial = new THREE.PointsMaterial({
448:     size: 0.8,
449:     color: 0xffffff,
450:     transparent: true,
451:     opacity: 0.25
452: });
453: starsSystem = new THREE.Points(starsGeometry, starsMaterial);
454: scene.add(starsSystem);
455:
456: // Animation loop
457: let clock = new THREE.Clock();
458: const animate = () => {
459:     animationFrameId = requestAnimationFrame(animate);
460:
461:     const time = clock.getElapsedTime();
462:
463:     // Rotate holographic core globe
464:     globeGroup.rotation.y = time * 0.05;
465:     globeGroup.rotation.x = time * 0.015;
466:
467:     // Orbiting data rings
468:     ringsData.forEach(ring => {
469:         ring.mesh.rotation.x += ring.speedX;
470:         ring.mesh.rotation.y += ring.speedY;
471:         ring.mesh.rotation.z += ring.speedZ;
472:     });
473:
474:     if (particleSystem) {
475:         particleSystem.rotation.y = time * 0.02;
476:     }
477:     if (linesSystem) {
478:         linesSystem.rotation.y = time * 0.02;
479:     }
480:     if (starsSystem) {
481:         starsSystem.rotation.y = time * 0.005;
482:     }
483:
484:     // Dynamic breathing/pulsing
485:     const pulseFactor = Math.sin(time * 2.0) * 0.2 + 0.8; // oscillates 0.6 to 1.0
486:     particleMaterial.size = 2.8 * pulseFactor;
487:     linesMaterial.opacity = 0.12 * (Math.sin(time * 1.5) * 0.3 + 0.7);
488:
489:     // Mouse Parallax smooth transition (Interpolation)
490:     const targetX = mouseX * 22;
491:     const targetY = mouseY * 22;
492:
493:     camera.position.x += (targetX - camera.position.x) * 0.04;
494:     camera.position.y += (targetY - camera.position.y) * 0.04;
495:     camera.lookAt(scene.position);
496:
497:     renderer.render(scene, camera);
498: };
499:
```

```
500:   animate();
501: };
502:
503: const handleResize = () => {
504:   if (!canvasContainer.value || !camera || !renderer) return;
505:   const width = canvasContainer.value.clientWidth;
506:   const height = canvasContainer.value.clientHeight;
507:
508:   camera.aspect = width / height;
509:   camera.updateProjectionMatrix();
510:   renderer.setSize(width, height);
511: };
512:
513: const onMouseMove = (event) => {
514:   if (!canvasContainer.value) return;
515:   const rect = canvasContainer.value.getBoundingClientRect();
516:   mouseX = ((event.clientX - rect.left) / rect.width) * 2 - 1;
517:   mouseY = -((event.clientY - rect.top) / rect.height) * 2 + 1;
518: };
519:
520: const loadScript = (src, id) => {
521:   return new Promise((resolve, reject) => {
522:     if (document.getElementById(id)) {
523:       resolve();
524:       return;
525:     }
526:     const script = document.createElement('script');
527:     script.src = src;
528:     script.id = id;
529:     script.async = true;
530:     script.defer = true;
531:     script.onload = () => resolve();
532:     script.onerror = () => reject(new Error(`Failed to load script ${src}`));
533:     document.head.appendChild(script);
534:   });
535: };
536:
537: const turnstileWidgetId = ref(null);
538:
539: const renderTurnstile = () => {
540:   if (window.turnstile && !turnstileWidgetId.value) {
541:     const container = document.getElementById('turnstile-container');
542:     if (container) {
543:       try {
544:         turnstileWidgetId.value = window.turnstile.render('#turnstile-container', {
545:           sitekey: TURNSTILE_SITE_KEY,
546:           theme: 'dark',
547:           callback: (token) => {
548:             turnstileToken.value = token;
549:             errorMessage.value = '';
550:           },
551:           'expired-callback': () => {
552:             turnstileToken.value = '';
553:           },
554:           'error-callback': () => {
555:             turnstileToken.value = '';
556:           }
557:         });
558:       } catch (err) {
559:         console.error('Error rendering Turnstile widget:', err);
560:       }
561:     }
562:   }
```

```
563: };
564:
565: watch(isRegisterMode, async (newVal) => {
566:   if (!newVal) {
567:     await nextTick();
568:     renderTurnstile();
569:   }
570: });
571:
572: watch(showTurnstile, async (newVal) => {
573:   if (newVal) {
574:     await nextTick();
575:     renderTurnstile();
576:   }
577: });
578:
579: onMounted(async () => {
580:   initThree();
581:   fetchPublicKey();
582:   window.addEventListener('resize', handleResize);
583:   window.addEventListener('mousemove', onMouseMove);
584:
585:   // Define global callback for Turnstile
586:   window.onloadTurnstileCallback = () => {
587:     renderTurnstile();
588:   };
589:
590:   // 1. Load Google reCAPTCHA v3 script independently so failure does not block Turnstile
591:   loadScript(`https://www.google.com/recaptcha/api.js?render=${RECAPTCHA_SITE_KEY}`, 'recaptch ...
592:   .catch(err => {
593:     console.warn('Google reCAPTCHA v3 script failed to load. Operating in Turnstile-only mod ...
594:   });
595:
596:   // 2. Load Cloudflare Turnstile script independently
597:   if (window.turnstile) {
598:     renderTurnstile();
599:   } else {
600:     loadScript('https://challenges.cloudflare.com/turnstile/v0/api.js?onload=onloadTurnstileCa ...
601:     .catch(err => {
602:       console.error('Failed to load Cloudflare Turnstile script:', err);
603:       errorMessage.value = 'Security verification failed to load. Please check your internet ...
604:     });
605:   }
606: });
607:
608: onUnmounted(() => {
609:   window.removeEventListener('resize', handleResize);
610:   window.removeEventListener('mousemove', onMouseMove);
611:   cancelAnimationFrame(animationFrameId);
612:   if (renderer) {
613:     renderer.dispose();
614:   }
615:
616:   // Clean up script tags to prevent memory leaks
617:   const recaptchaScript = document.getElementById('recaptcha-script');
618:   if (recaptchaScript) recaptchaScript.remove();
619:   const turnstileScript = document.getElementById('turnstile-script');
620:   if (turnstileScript) turnstileScript.remove();
621:
622:   // Clean up any remaining recaptcha badge
623:   const badge = document.querySelector('.grecaptcha-badge');
624:   if (badge) badge.remove();
625: });
```

```

626: </script>
627:
628: <template>
629:   <div class="min-h-screen w-full flex bg-[#060913] text-white relative overflow-hidden font-s ...
630:
631:     <!-- 3D Visualization Pane (Left Side) -->
632:     <div ref="canvasContainer" class="hidden lg:block lg:w-[58%] h-screen relative bg-[#04060c] ...
633:       <!-- Overlay Info Details -->
634:       <div class="absolute top-10 left-10 z-10 flex items-center gap-3">
635:         <div class="bg-gradient-to-tr from-violet-600 to-indigo-600 p-2.5 rounded-xl shadow-lg ...
636:           <i class="fa-solid fa-tower-broadcast text-xl text-white"></i>
637:         </div>
638:         <div>
639:           <h2 class="text-xl font-bold tracking-tight text-white">WEB SITE MONITORING SYSTEM</ ...
640:           <p class="text-xs text-gray-400">Observability Gateway & Observational Control</p>
641:         </div>
642:       </div>
643:
644:       <div class="absolute bottom-10 left-10 z-10 max-w-md">
645:         <h3 class="text-lg font-bold text-violet-400 mb-1 flex items-center gap-2">
646:           <span class="inline-block w-2.5 h-2.5 rounded-full bg-emerald-400 pulse-green"></spa ...
647:           Enterprise Observatory Active
648:         </h3>
649:         <p class="text-xs text-gray-400 leading-relaxed">
650:           Secured with 256-bit HMAC JWT tokens, state-synchronized database logging, and auto- ...
651:         </p>
652:       </div>
653:     </div>
654:
655:     <!-- Obsidian Glassmorphism Auth Card (Right Side) -->
656:     <div class="w-full lg:w-[42%] flex items-center justify-center p-6 md:p-12 bg-gradient-to- ...
657:
658:       <!-- Decorative Orbs -->
659:       <div class="absolute top-[20%] right-[-10%] w-[300px] h-[300px] glow-orb-1 rounded-full ...
660:       <div class="absolute bottom-[20%] left-[-10%] w-[300px] h-[300px] glow-orb-2 rounded-ful ...
661:
662:       <!-- Obsidian Card -->
663:       <div :class="[
664:         'w-full max-w-md glass-card rounded-3xl p-8 border border-white/10 shadow-2xl relative ...
665:         isShaking ? 'shake-animation border-rose-500/50' : ''
666:       ]">
667:
668:         <!-- Header -->
669:         <header class="text-center mb-8">
670:           <h1 class="text-2xl font-bold text-white tracking-tight flex items-center justify-ce ...
671:             {{ isRegisterMode ? 'Create Account in' : 'Sign In to' }} <span class="bg-gradient ...
672:           </h1>
673:           <p class="text-xs text-gray-400 mt-2">
674:             {{ isRegisterMode ? 'Register a new telemetry operator account' : 'Enter credentia ...
675:           </p>
676:         </header>
677:
678:         <!-- Notification Banners -->
679:         <div v-if="errorMessage" class="mb-5 p-3 rounded-xl bg-rose-950/40 border border-rose- ...
680:           <i class="fa-solid fa-triangle-exclamation text-rose-400 text-sm"></i>
681:           <span>{{ errorMessage }}</span>
682:         </div>
683:
684:         <div v-if="successMessage" class="mb-5 p-3 rounded-xl bg-emerald-950/40 border border- ...
685:           <i class="fa-solid fa-circle-check text-emerald-400 text-sm"></i>
686:           <span>{{ successMessage }}</span>
687:         </div>
688:

```

```

689:     <!-- Credentials Form -->
690:     <form @submit.prevent="handleSubmit" class="flex flex-col gap-4">
691:         <div>
692:             <label for="username" class="block text-xs font-semibold text-gray-400 mb-1.5 uppe ...
693:                 {{ isRegisterMode ? 'Username' : 'Username or Email' }}
694:             </label>
695:             <div class="relative">
696:                 <span class="absolute inset-y-0 left-0 flex items-center pl-3.5 text-gray-500">
697:                     <i class="fa-solid fa-user text-xs"></i>
698:                 </span>
699:                 <input
700:                     type="text"
701:                     id="username"
702:                     required
703:                     v-model="usernameOrEmail"
704:                     :placeholder="isRegisterMode ? 'telemetry_operator' : 'admin'"
705:                     class="w-full glass-input pl-10 pr-4 py-2.5 rounded-xl text-gray-200 text-sm f ...
706:                 >
707:             </div>
708:         </div>
709:
710:         <!-- Email Field (Register Only) -->
711:         <div v-if="isRegisterMode">
712:             <label for="email" class="block text-xs font-semibold text-gray-400 mb-1.5 upperca ...
713:             <div class="relative">
714:                 <span class="absolute inset-y-0 left-0 flex items-center pl-3.5 text-gray-500">
715:                     <i class="fa-solid fa-envelope text-xs"></i>
716:                 </span>
717:                 <input
718:                     type="email"
719:                     id="email"
720:                     required
721:                     v-model="email"
722:                     placeholder="operator@wsms.local"
723:                     class="w-full glass-input pl-10 pr-4 py-2.5 rounded-xl text-gray-200 text-sm f ...
724:                 >
725:             </div>
726:         </div>
727:
728:         <div>
729:             <label for="password" class="block text-xs font-semibold text-gray-400 mb-1.5 uppe ...
730:             <div class="relative">
731:                 <span class="absolute inset-y-0 left-0 flex items-center pl-3.5 text-gray-500">
732:                     <i class="fa-solid fa-lock text-xs"></i>
733:                 </span>
734:                 <input
735:                     type="password"
736:                     id="password"
737:                     required
738:                     v-model="password"
739:                     placeholder=""
740:                     class="w-full glass-input pl-10 pr-4 py-2.5 rounded-xl text-gray-200 text-sm f ...
741:                 >
742:             </div>
743:         </div>
744:
745:         <!-- Confirm Password (Register Only) -->
746:         <div v-if="isRegisterMode">
747:             <label for="confirmPassword" class="block text-xs font-semibold text-gray-400 mb-1 ...
748:             <div class="relative">
749:                 <span class="absolute inset-y-0 left-0 flex items-center pl-3.5 text-gray-500">
750:                     <i class="fa-solid fa-lock text-xs"></i>
751:                 </span>

```

```

752:         <input
753:             type="password"
754:             id="confirmPassword"
755:             required
756:             v-model="confirmPassword"
757:             placeholder=""
758:             class="w-full glass-input pl-10 pr-4 py-2.5 rounded-xl text-gray-200 text-sm f ...
759:         >
760:     </div>
761: </div>
762:
763: <!-- Cloudflare Turnstile Container -->
764: <div v-show="showTurnstile && !isRegisterMode" class="my-3 flex flex-col items-cente ...
765:     <div v-if="!turnstileToken && !turnstileWidgetId" class="text-xs text-gray-500 fle ...
766:         <i class="fa-solid fa-circle-notch animate-spin text-violet-400 text-sm"></i>
767:         <span>Securing connection...</span>
768:     </div>
769:     <div id="turnstile-container"></div>
770: </div>
771:
772: <!-- Submit Button -->
773: <button
774:     type="submit"
775:     :disabled="loading"
776:     class="w-full mt-4 bg-gradient-to-r from-violet-600 to-indigo-600 hover:from-viole ...
777: >
778:     <i class="fa-solid fa-arrows-spin animate-spin" v-if="loading"></i>
779:     <i class="fa-solid fa-shield-halved" v-else></i>
780:     {{ isRegisterMode ? 'Register Operator Account' : 'Authorize Session' }}
781: </button>
782: </form>
783:
784: <!-- Toggle Mode Link -->
785: <p class="text-xs text-gray-400 text-center mt-5">
786:     {{ isRegisterMode ? 'Already have an account?' : "Don't have an account?" }}
787:     <button
788:         @click="isRegisterMode = !isRegisterMode; errorMessage = ''; successMessage = '';"
789:         class="text-violet-400 hover:text-violet-300 font-semibold ml-1 focus:outline-none ...
790:     >
791:         {{ isRegisterMode ? 'Sign In' : 'Register Now' }}
792:     </button>
793: </p>
794:
795: <!-- Divider -->
796: <div class="flex items-center my-6">
797:     <div class="flex-1 border-t border-white/5"></div>
798:     <span class="px-3 text-[10px] text-gray-500 font-bold uppercase tracking-wider">or s ...
799:     <div class="flex-1 border-t border-white/5"></div>
800: </div>
801:
802: <!-- Social logins (Google & GitHub) -->
803: <div class="grid grid-cols-2 gap-3">
804:     <button
805:         @click="handleGoogleStub"
806:         class="bg-white/5 hover:bg-white/10 text-gray-200 border border-white/5 py-2.5 rou ...
807:     >
808:         
811:
812:     <button
813:         @click="handleGithubStub"
814:         class="bg-white/5 hover:bg-white/10 text-gray-200 border border-white/5 py-2.5 rou ...

```

```
815:         >
816:         <i class="fa-brands fa-github text-sm text-white"></i>
817:         GitHub
818:     </button>
819: </div>
820:
821: <!-- Footer -->
822: <footer class="mt-8 text-center text-[10px] text-gray-500 leading-relaxed">
823:     By signing in, you agree to comply with standard security and data handling policies ...
824:     Default admin credentials are listed in the onboarding guide.
825: </footer>
826:
827: </div>
828: </div>
829: </div>
830: </template>
831:
832: <style scoped>
833: .glass-input {
834:     background: rgba(255, 255, 255, 0.02);
835:     border: 1px solid rgba(255, 255, 255, 0.05);
836:     transition: all 0.2s ease;
837: }
838: .glass-input:focus {
839:     background: rgba(255, 255, 255, 0.04);
840:     border-color: rgba(139, 92, 246, 0.5);
841:     box-shadow: 0 0 10px rgba(139, 92, 246, 0.15);
842: }
843:
844: .shake-animation {
845:     animation: shake 0.5s ease-in-out;
846: }
847:
848: @keyframes shake {
849:     0%, 100% { transform: translateX(0); }
850:     20%, 60% { transform: translateX(-6px); }
851:     40%, 80% { transform: translateX(6px); }
852: }
853:
854: .glow-orb-1 {
855:     background: radial-gradient(circle, rgba(139, 92, 246, 0.15) 0%, rgba(0,0,0,0) 70%);
856:     filter: blur(40px);
857: }
858: .glow-orb-2 {
859:     background: radial-gradient(circle, rgba(79, 70, 229, 0.15) 0%, rgba(0,0,0,0) 70%);
860:     filter: blur(40px);
861: }
862:
863: .pulse-green {
864:     box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.5);
865:     animation: pulse-green-anim 2s infinite;
866: }
867: @keyframes pulse-green-anim {
868:     0% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.5); }
869:     70% { transform: scale(1); box-shadow: 0 0 0 6px rgba(16, 185, 129, 0); }
870:     100% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(16, 185, 129, 0); }
871: }
872: </style>
```